# AssignmentNumber:9.4

**Name:J.Mukhesh Hall:2303a51813**

**Batch-26**

| | | |
|---|---|---|
| 1 | **Lab 9 – Documentation Generation: Automatic Documentation and Code Comments**<br><br>**Lab Objectives**<br>• To use AI-assisted coding tools for generating Python documentation and code comments.<br>• To apply zero-shot, few-shot, and context-based prompt engineering for documentation creation.<br>• To practice generating and refining docstrings, inline comments, and module-level documentation.<br>• To compare outputs from different prompting styles for quality analysis.<br><br>**Lab Outcomes**<br>• Generate structured code documentation using AI tools<br>• Apply appropriate documentation styles to different code contexts<br>• Improve code readability through selective commenting<br>• Convert informal developer comments into professional documentation<br>• Analyze and refine AI-generated documentation | Week 5 |
| | **Task 1: Auto-Generating Function Documentation in a Shared Codebase**<br><br>## Scenario<br>You have joined a development team where several utility functions are already implemented, but the code lacks proper documentation. New team members are struggling to understand how these functions should be used.<br><br>## Task Description<br>You are given a Python script containing multiple functions without any docstrings.<br><br>Using an AI-assisted coding tool:<br><br>• Ask the AI to automatically generate **Google-style function docstrings** for each function<br>• Each docstring should include:<br>  ○ A brief description of the function<br>  ○ Parameters with data types | |

- Return values
- At least one example usage (if applicable)

Experiment with different prompting styles (zero-shot or context-based) to observe quality differences.

## Expected Outcome

- A Python script with well-structured Google-style docstrings
- Docstrings that clearly explain function behavior and usage
- Improved readability and usability of the codebase

Promt:

##Generate Google-style docstrings for the following Python functions.

#Include:

#- Brief description

#- Parameters with data types

#- Return value

#- Example usage

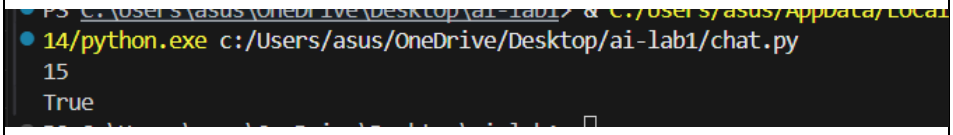
Code:

```
def add(a, b):
    """
    Adds two numbers and returns the result.

    Args:
        a (int or float): First number.
        b (int or float): Second number.
```

```
    Returns:
        int or float: Sum of the two numbers.

    Example:
        >>> add(2, 3)
        5
    """
    return a + b


add(5, 10)
def is_even(n):
    """
    Checks whether a number is even.

    Args:
        n (int): Input number.

    Returns:
        bool: True if even, False otherwise.

    Example:
        >>> is_even(6)
        True
```

```
    """

    return n % 2 == 0

# Example usage of the add function

print(add(5, 10))  # Example usage of the add function

print(is_even(4))  # Example usage of the is_even function
```

Output:




**Task 2: Enhancing Readability Through AI-Generated Inline Comments**

## Scenario

A Python program contains complex logic that works correctly but is difficult to understand at first glance. Future maintainers may find it hard to debug or extend this code.

## Task Description

You are provided with a Python script containing:

- Loops
- Conditional logic
- Algorithms (such as Fibonacci sequence, sorting, or searching)

Use AI assistance to:

- Automatically insert **inline comments only for complex or non-obvious logic**
- Avoid commenting on trivial or self-explanatory syntax

The goal is to improve clarity without cluttering the code.

## Expected Outcome

- A Python script with concise, meaningful inline comments
- Comments that explain *why* the logic exists, not *what* Python syntax does
- Noticeable improvement in code readability

Promt:

#Add inline comments only for complex or non-obvious logic.

#Do not comment on simple or self-explanatory syntax.

Code:

```
def fibonacci(n):
    a, b = 0, 1  # Initialize first two Fibonacci numbers
    for _ in range(n):
        # Move forward in the Fibonacci sequence
        a, b = b, a + b
    return a
print(fibonacci(10))  # Output: 55
```

Output:

```
PS C:\Users\asus\OneDrive\Desktop\ai-lab1> & C:/Users/asus/AppData/Local/Progra
14/python.exe c:/Users/asus/OneDrive/Desktop/ai-lab1/chat.py
55
PS C:\Users\asus\OneDrive\Desktop\ai-lab1>
```

**Task 3: Generating Module-Level Documentation for a Python Package**

## Scenario

Your team is preparing a Python module to be shared internally (or uploaded to a repository). Anyone opening the file should immediately understand its purpose and structure.

## Task Description

Provide a complete Python module to an AI tool and instruct it to automatically generate a **module-level docstring** at the top of the file that includes:

- The purpose of the module
- Required libraries or dependencies
- A brief description of key functions and classes
- A short example of how the module can be used

Focus on clarity and professional tone.

## Expected Outcome

- A well-written multi-line module-level docstring
- Clear overview of what the module does and how to use it
- Documentation suitable for real-world projects or repositories

Promt:

```
#Generate a professional module-level docstring.
#Include:
#- Purpose of the module
#- Dependencies
#- Description of functions
#- Example usage
```

Code:

```
"""
math_utils.py

This module provides basic mathematical utility functions.

Dependencies:
    None

Functions:
    add(a, b): Returns the sum of two numbers.
    subtract(a, b): Returns the difference of two numbers.

Example:
    >>> from math_utils import add
    >>> add(4, 5)
    9
"""


def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
print(add(4, 5))
print(subtract(10, 3))
```

Output:

```
PS C:\Users\asus\OneDrive\Desktop\ai-lab1> & C:/Users/asus/AppData/Local/Programs/Python/P
14/python.exe c:/Users/asus/OneDrive/Desktop/ai-lab1/chat.py
9
7
PS C:\Users\asus\OneDrive\Desktop\ai-lab1> & C:/Users/asus/AppData/Local/Programs/Python/P
```

**Task 4: Converting Developer Comments into Structured Docstrings**

## Scenario

In a legacy project, developers have written long explanatory comments inside functions instead of proper docstrings. The team now wants to standardize documentation.

## Task Description

You are given a Python script where functions contain detailed inline comments explaining their logic.
Use AI to:

- Automatically convert these comments into structured **Google-style or NumPy-style docstrings**
- Preserve the original meaning and intent of the comments
- Remove redundant inline comments after conversion

## Expected Outcome

- Functions with clean, standardized docstrings
- Reduced clutter inside function bodies
- Improved consistency across the codebase

Promt:
#Convert inline developer comments into Google-style docstrings.
#Removeredundant comments after conversion.
#Preserve the original meaning.

Code:

```
def divide(a, b):
    """
    Divides one number by another.

    Args:
        a (int or float): Numerator.
        b (int or float): Denominator.

    Returns:
        float or None: Division result or None if division by zero.
    """
    if b == 0:
        return None
    return a / b
```

Output:

```
PS C:\Users\asus\OneDrive\Desktop\ai-lab1> & C:/Users/asus/AppData/Local/Programs
14/python.exe c:/Users/asus/OneDrive/Desktop/ai-lab1/chat.py
5.0
None
```

**Task 5: Building a Mini Automatic Documentation Generator**

## Scenario

Your team wants a simple internal tool that helps developers start documenting new Python files quickly, without writing documentation from scratch.

## Task Description

Design a small Python utility that:

- Reads a given `.py` file
- Automatically detects:
  - Functions
  - Classes
- Inserts **placeholder Google-style docstrings** for each detected function or class

AI tools may be used to assist in generating or refining this utility.
Note: The goal is **documentation scaffolding**, not perfect documentation.

## Expected Outcome

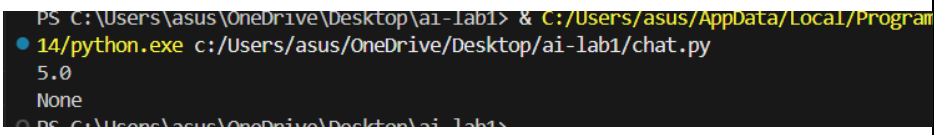
- A working Python script that processes another `.py` file
- Automatically inserted placeholder docstrings
- Clear demonstration of how AI can assist in documentation automation

Promt:
#Create a Python utility that detects functions and classes
and inserts placeholder Google-style docstrings.
Focus on documentation scaffolding.

Code:

```
import ast
def generate_docstrings(file_path):
    with open(file_path, "r") as file:
        tree = ast.parse(file.read())
    for node in tree.body:
        if isinstance(node, ast.FunctionDef):
            print(f'Function: {node.name}')
            print("""")
            print("Description:")
            print("\nArgs:")
            for arg in node.args.args:
                print(f"    {arg.arg} (type): description")
            print("\nReturns:")
            print("    type: description")
            print("""")
            print()
```

Output:




**Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots**